

Name – Earl Pratik Sinha

Roll – 2024AD05449

1. Setup & Installation Instructions

This repository contains an end-to-end MLOps pipeline for predicting heart disease using a FastAPI backend, Docker containerization, and a Kubernetes deployment.

Prerequisites

Ensure the following tools are installed on your system before proceeding:

- **Git** (for version control)
- **Python 3.10+**
- **Docker Desktop** (running)
- **Minikube & Kubectl** (for local Kubernetes deployment)

Option A: Local Setup (Development)

1. Clone the repository:

Bash

```
git clone https://github.com/earlpratiksinha/heart-disease-mlops.git
```

```
cd heart-disease-mlops
```

2. Create and activate a virtual environment:

Bash

```
python -m venv venv
```

On Windows:

```
.\venv\Scripts\activate
```

On macOS/Linux:

```
source venv/bin/activate
```

3. Install dependencies:

Bash

```
pip install -r requirements.txt
```

4. Fetch data and train the model:

Bash

```
python src/download_data.py
```

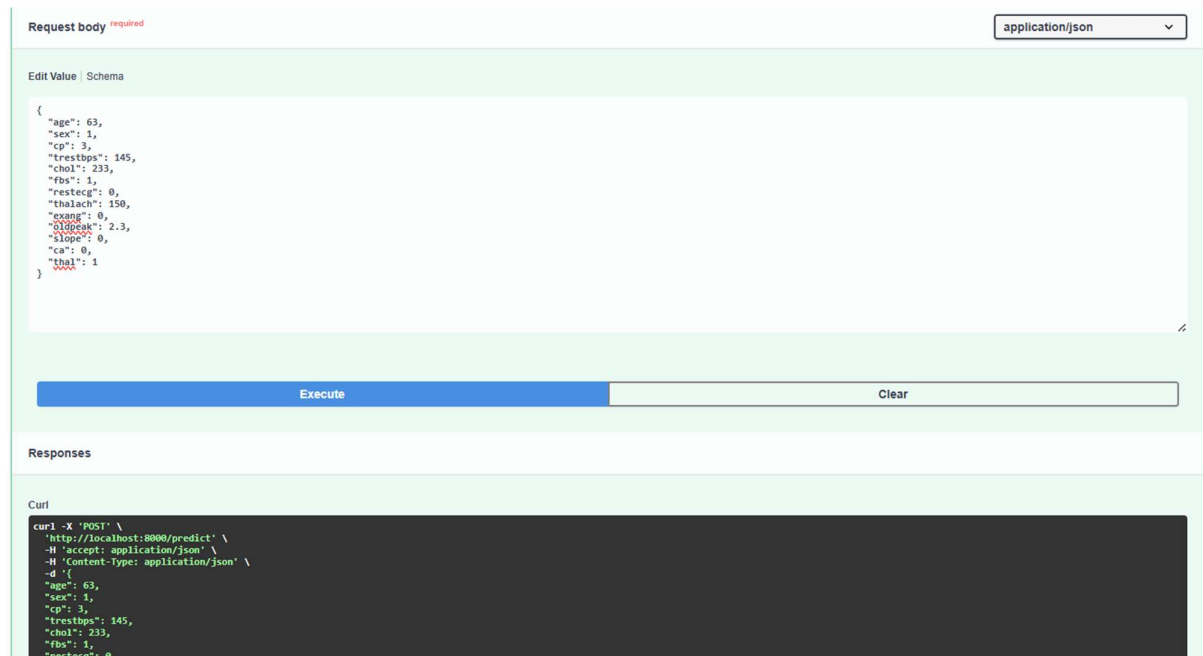
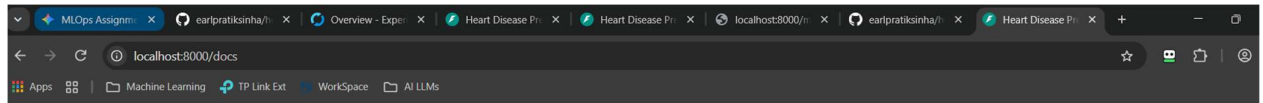
```
python src/train.py
```

5. Run the API locally:

Bash

uvicorn src.api:app --reload

The API will be available at <http://localhost:8000/docs>.



```

{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
}

```

Request URL

http://localhost:8000/predict

Server response

Code	Details
200	Response body <pre> { "prediction": 0, "confidence": 0.74 } </pre> Response headers <pre> content-length: 34 content-type: application/json date: Sat, 11 Jul 2026 13:35:24 GMT server: uvicorn </pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls: Accept header

Example Value | Schema

```

"string"

```

Option B: Docker Setup (Containerized)

1. Build the Docker image:

Bash

docker build -t heart-disease-api .

```

(venv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops> docker build -t heart-disease-api .
[+] Building 261.2s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 665B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.10-slim@sha256:e5300dc028a26a34a19337a57602955a2510e22abeb176edd6de6cd2cc927dd4
=> => resolve docker.io/library/python:3.10-slim@sha256:e5300dc028a26a34a19337a57602955a2510e22abeb176edd6de6cd2cc927dd4
=> [internal] load build context
=> => transferring context: 7.11kB
=> CACHED [2/6] WORKDIR /app
=> [3/6] COPY requirements.txt .
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> [5/6] COPY src/ /app/src/
=> [6/6] COPY models/ /app/models/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5095999e7379763edcaa77439ab41f7890b33b174ccc9a8c60091429217287d8
=> => exporting config sha256:71b0edc9ee1e2dbaa5615e82625df3ad5600e1866b5d9996155eb2d4d5f51fb3
=> => exporting attestation manifest sha256:c9c5f96ff4d3061632b62dec260693cc58fa15241d377f8a90a898b7d578ce39
=> => exporting manifest list sha256:6046259dca1ef1539ae41fb08b90c38a197d49c4876ec016ea697b5a4f187a1c
=> => naming to docker.io/library/heart-disease-api:latest
=> => unpacking to docker.io/library/heart-disease-api:latest

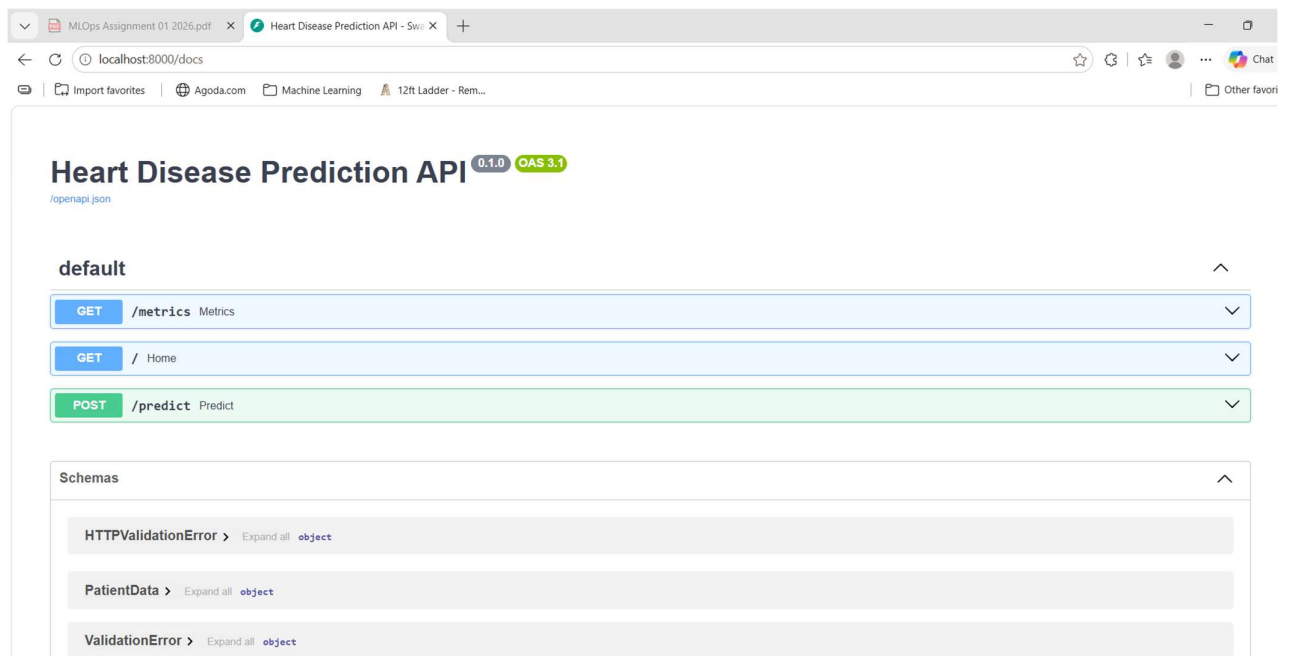
```

2. Run the container:

Bash

docker run -p 8000:8000 heart-disease-api

```
(venv) PS C:\Users\Earl Pratik Sinha\Desktop\WLOps Assignment\heart-disease-mlops> docker run -p 8000:8000 heart-disease-api
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.9.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.9.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.9.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.9.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
2026-07-11 13:50:17,279 - src.api - INFO - Model loaded successfully.
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```



Option C: Kubernetes Setup (Production/Minikube)

1. Start Minikube:

Bash

```
minikube start
```

2. Load the Docker image into Minikube's local registry:

Bash

```
minikube image load heart-disease-api:latest
```

3. Deploy the application and service:

Bash

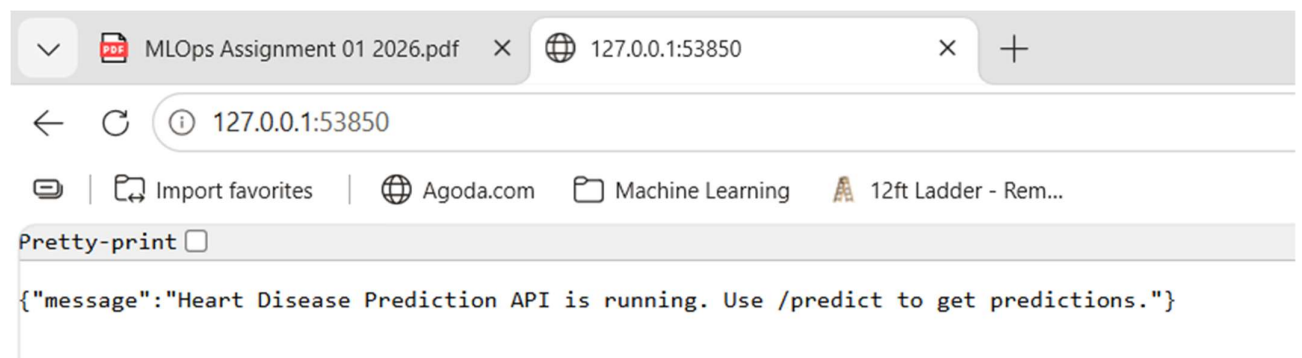
```
kubectl apply -f k8s_deployment.yaml
```

4. Verify the pods are running:

```
kubectl get pods
```



minikube service heart-disease-api-service



Heart Disease Prediction API 0.1.0 OAS 3.1

/openapi.json

default

GET	/ Home
POST	/predict Predict

Schemas

HTTPValidationError > Expand all object

PatientData > Expand all object

ValidationError > Expand all object

GET	/ Home
POST	/predict Predict

Parameters

Try it out

No parameters

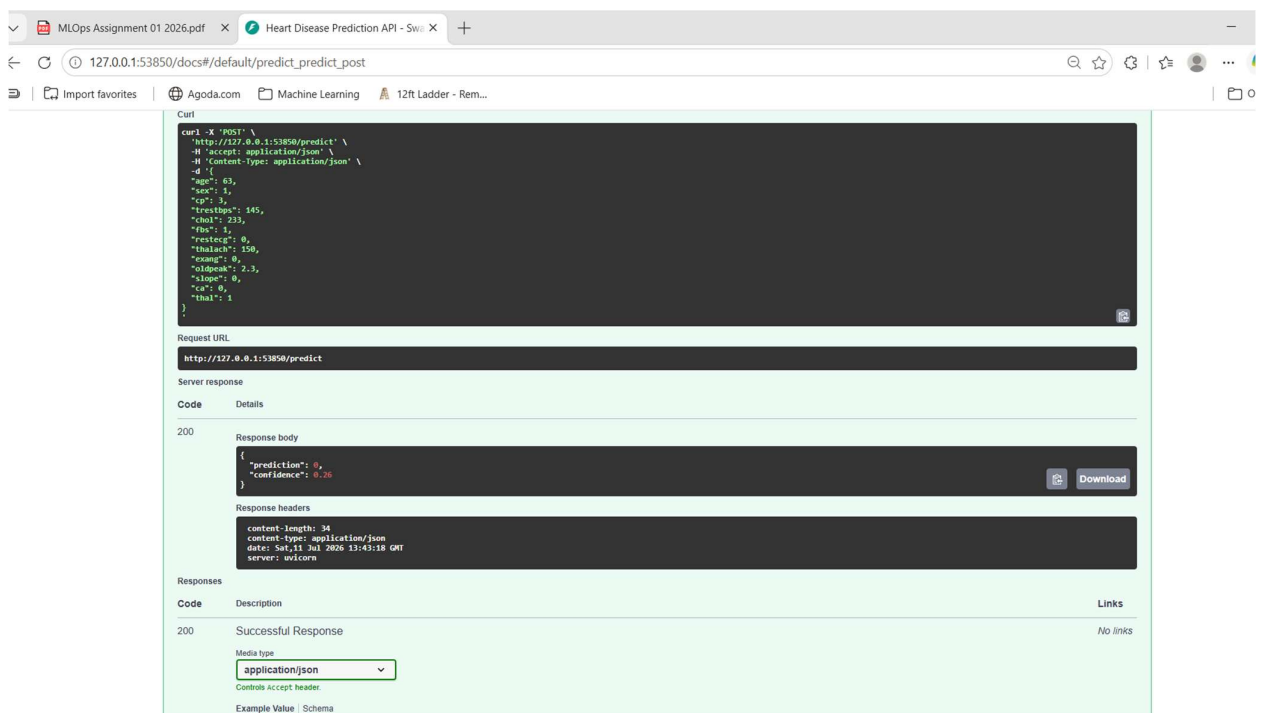
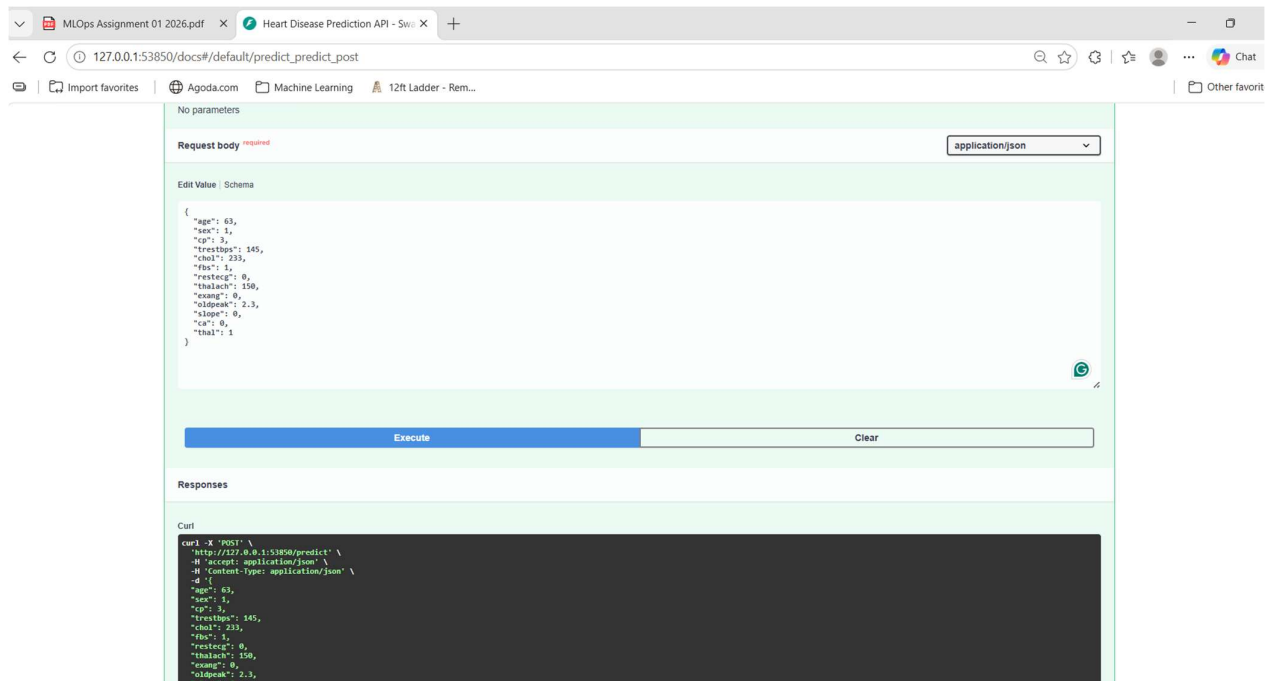
Request body required

application/json

Example Value | Schema

```
{  "age": 0,  "sex": 0,  "cp": 0,  "trestbps": 0,  "chol": 0,  "fbs": 0,  "restecg": 0,  "thalach": 0,  "exang": 0,  "oldpeak": 0,  "slope": 0,  "ca": 0,  "thal": 0}
```

Responses



This command will automatically tunnel and open the API in your default web browser.

2. System Architecture & MLOps Pipeline

This project implements a fully automated end-to-end machine learning lifecycle:

- **Data Ingestion & Processing:** The `download_data.py` script automatically fetches the Heart Disease dataset from the UCI Machine Learning Repository, engineers the target variable, handles missing values via median imputation, and outputs a clean CSV.

- **Model Training:** The `train.py` script splits the data, trains a machine learning model, evaluates its accuracy, and serializes the best-performing model using `joblib`.
- **API Development:** A robust REST API built with **FastAPI** serves the model. It includes data validation via `Pydantic` and returns both the prediction and the model's confidence score.
- **Containerization:** The application is packaged into an isolated, reproducible environment using **Docker**, ensuring consistency across development and production.
- **CI/CD Automation:** A **GitHub Actions** workflow is configured to trigger on every push to the main branch. It automatically sets up the Python environment, runs the data pipeline, trains the model, executes `pytest` unit tests, and builds the Docker image.
- **Production Deployment:** The containerized application is deployed to a local **Kubernetes** cluster (Minikube). The deployment configuration (`k8s_deployment.yaml`) manages a replica set of 2 pods for high availability and exposes the application via a `NodePort` service.
- **Observability:** The API integrates `prometheus-fastapi-instrumentator` and Python's logging module to track HTTP request metrics, latency, and prediction events at the `/metrics` endpoint.

3. API Documentation

POST /predict

Accepts patient health metrics and returns a heart disease prediction.

- **Content-Type:** `application/json`
- **Example Payload:**

JSON

```
{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
```



```
"thal": 1
}
```

- **Example Response:**

JSON

```
{
  "prediction": 1,
  "confidence": 0.85
}
```

GET /metrics

Exposes system and API metrics (requests, latency, memory usage) in plain text format for Prometheus scraping.

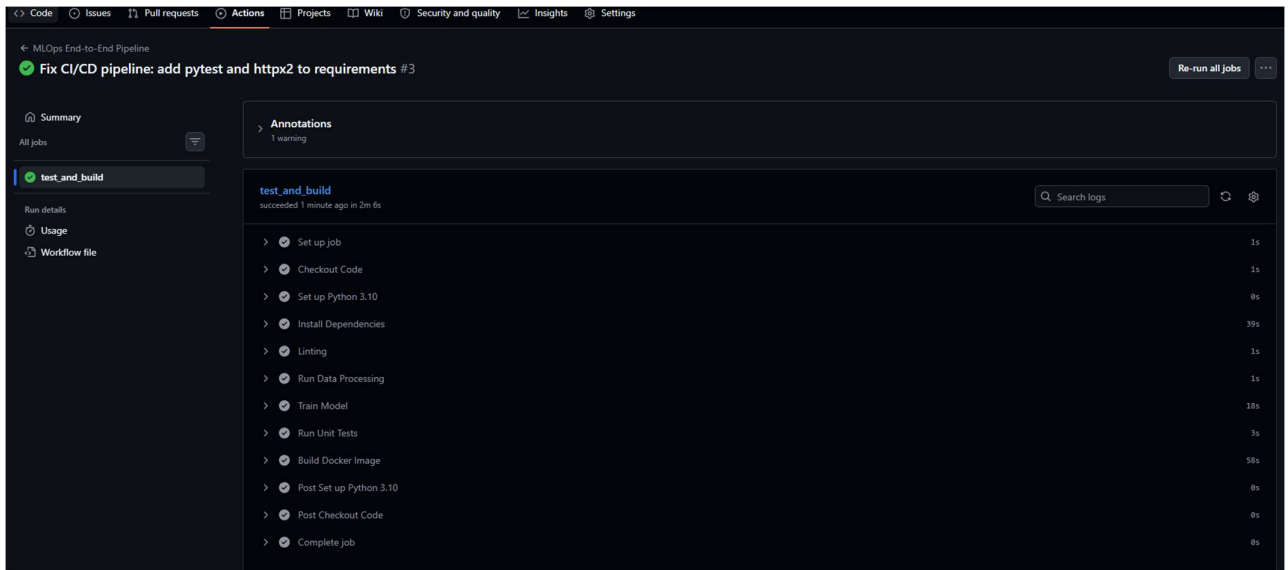
4. Proof of Execution

Public GitHub Repository: <https://github.com/earlpratiksinha/heart-disease-mlops>

4.1. CI/CD Pipeline Success

The screenshot shows the GitHub Actions interface for the repository 'earlpratiksinha / heart-disease-mlops'. The 'All workflows' tab is selected, displaying a list of workflow runs. The left sidebar shows the 'Actions' section with a 'New workflow' button and a list of workflow categories: 'All workflows', 'MLOps End-to-End Pipeline', 'Management', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area shows '3 workflow runs' with a search bar 'Filter workflow runs'. The runs are listed in a table with columns for Workflow, Event, Status, Branch, and Actor. The first run is 'Fix CI/CD pipeline: add pytest and httpx2 to requirements' (green status, 3 minutes ago). The second run is 'Fix CI/CD pipeline: add data cleaning to download script' (red status, 6 minutes ago). The third run is 'Complete MLOps pipeline setup: data, training, API, tests, Docker, an...' (red status, 9 minutes ago).

Workflow	Event	Status	Branch	Actor
Fix CI/CD pipeline: add pytest and httpx2 to requirements	MLOps End-to-End Pipeline #3: Commit 3334d15 pushed by earlpratiksinha	Success	main	earlpratiksinha
Fix CI/CD pipeline: add data cleaning to download script	MLOps End-to-End Pipeline #2: Commit bba1e5e pushed by earlpratiksinha	Failure	main	earlpratiksinha
Complete MLOps pipeline setup: data, training, API, tests, Docker, an...	MLOps End-to-End Pipeline #1: Commit 4dc90fa pushed by earlpratiksinha	Failure	main	earlpratiksinha



4.2. Kubernetes Deployment

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(minv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops> minikube start
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.50 ...
Downloading Kubernetes v1.35.1 preload ...
> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 10.10 M
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 8.79 Mi
Creating docker container (CPUs=2, Memory=3072MB) ...
Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
Done! kubect1 is now configured to use "minikube" cluster and "default" namespace by default
(minv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops> minikube image load heart-disease-api:latest
(minv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops> kubectl apply -f k8s_deployment.yaml
deployment.apps/heart-disease-api-deployment created
service/heart-disease-api-service created
(minv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops> kubectl get pods
NAME                                READY STATUS RESTARTS AGE
heart-disease-api-deployment-7c4d9b74b6-88tk2 1/1 Running 0 5s
heart-disease-api-deployment-7c4d9b74b6-mdhhz 1/1 Running 0 5s
(minv) PS C:\Users\Earl Pratik Sinha\Desktop\MLops Assignment\heart-disease-mlops>
```

4.3. Prometheus Monitoring

```
MLops Assignment Tool | Fix CI/CD pipeline: add pyt | Overview - Experiment 1 - | Heart Disease Prediction AI | Heart Disease Prediction AI | localhost:8000/metrics

localhost:8000/metrics

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 974.0
python_gc_objects_collected_total{generation="1"} 62.0
python_gc_objects_collected_total{generation="2"} 16.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 140.0
python_gc_collections_total{generation="1"} 12.0
python_gc_collections_total{generation="2"} 1.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="13",patchlevel="5",version="3.13.5"} 1.0
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{handler="/metrics",method="GET",status="2xx"} 1.0
http_requests_total{handler="none",method="GET",status="4xx"} 1.0
# HELP http_requests_created Total number of requests by method, status and handler.
# TYPE http_requests_created gauge
http_requests_created{handler="/metrics",method="GET",status="2xx"} 1.7837753058197167e+09
http_requests_created{handler="none",method="GET",status="4xx"} 1.783775306004615e+09
# HELP http_request_size_bytes Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes summary
http_request_size_bytes_count{handler="/metrics"} 1.0
http_request_size_bytes_sum{handler="/metrics"} 0.0
http_request_size_bytes_count{handler="none"} 1.0
http_request_size_bytes_sum{handler="none"} 0.0
# HELP http_request_size_bytes_created Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes_created gauge
http_request_size_bytes_created{handler="/metrics"} 1.7837753058197591e+09
http_request_size_bytes_created{handler="none"} 1.783775306004679e+09
# HELP http_response_size_bytes Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes summary
http_response_size_bytes_count{handler="/metrics"} 1.0
http_response_size_bytes_sum{handler="/metrics"} 3572.0
http_response_size_bytes_count{handler="none"} 1.0
http_response_size_bytes_sum{handler="none"} 22.0
# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge

# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge
http_response_size_bytes_created{handler="/metrics"} 1.783775305819816e+09
http_response_size_bytes_created{handler="none"} 1.783775306004784e+09
# HELP http_request_duration_highr_seconds Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds histogram
http_request_duration_highr_seconds_bucket{le="0.01"} 1.0
http_request_duration_highr_seconds_bucket{le="0.025"} 2.0
http_request_duration_highr_seconds_bucket{le="0.05"} 2.0
http_request_duration_highr_seconds_bucket{le="0.075"} 2.0
http_request_duration_highr_seconds_bucket{le="0.1"} 2.0
http_request_duration_highr_seconds_bucket{le="0.25"} 2.0
http_request_duration_highr_seconds_bucket{le="0.5"} 2.0
http_request_duration_highr_seconds_bucket{le="0.75"} 2.0
http_request_duration_highr_seconds_bucket{le="1.0"} 2.0
http_request_duration_highr_seconds_bucket{le="1.5"} 2.0
http_request_duration_highr_seconds_bucket{le="2.0"} 2.0
http_request_duration_highr_seconds_bucket{le="2.5"} 2.0
http_request_duration_highr_seconds_bucket{le="3.0"} 2.0
http_request_duration_highr_seconds_bucket{le="3.5"} 2.0
http_request_duration_highr_seconds_bucket{le="4.0"} 2.0
http_request_duration_highr_seconds_bucket{le="4.5"} 2.0
http_request_duration_highr_seconds_bucket{le="5.0"} 2.0
http_request_duration_highr_seconds_bucket{le="7.5"} 2.0
http_request_duration_highr_seconds_bucket{le="10.0"} 2.0
http_request_duration_highr_seconds_bucket{le="30.0"} 2.0
http_request_duration_highr_seconds_bucket{le="60.0"} 2.0
http_request_duration_highr_seconds_bucket{le="+Inf"} 2.0
http_request_duration_highr_seconds_count 2.0
http_request_duration_highr_seconds_sum 0.015254599999934726
# HELP http_request_duration_highr_seconds_created Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds_created gauge
http_request_duration_highr_seconds_created 1.7837753056137178e+09
# HELP http_request_duration_seconds Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{handler="/metrics",le="0.1",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="0.5",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="1.0",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="+Inf",method="GET"} 1.0
http_request_duration_seconds_count{handler="/metrics",method="GET"} 1.0
http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.013214900000093621
http_request_duration_seconds_bucket{handler="none",le="0.1",method="GET"} 1.0

http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.013214900000093621
http_request_duration_seconds_bucket{handler="none",le="0.1",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="none",le="0.5",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="none",le="1.0",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="none",le="+Inf",method="GET"} 1.0
http_request_duration_seconds_count{handler="none",method="GET"} 1.0
http_request_duration_seconds_sum{handler="none",method="GET"} 0.0020396999998411047
# HELP http_request_duration_seconds_created Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds_created gauge
http_request_duration_seconds_created{handler="/metrics",method="GET"} 1.7837753058198745e+09
http_request_duration_seconds_created{handler="none",method="GET"} 1.7837753060048678e+09
```

Link to code repository - <https://github.com/earlpratiksinha/heart-disease-mlops>

5. EDA and Modelling Choices

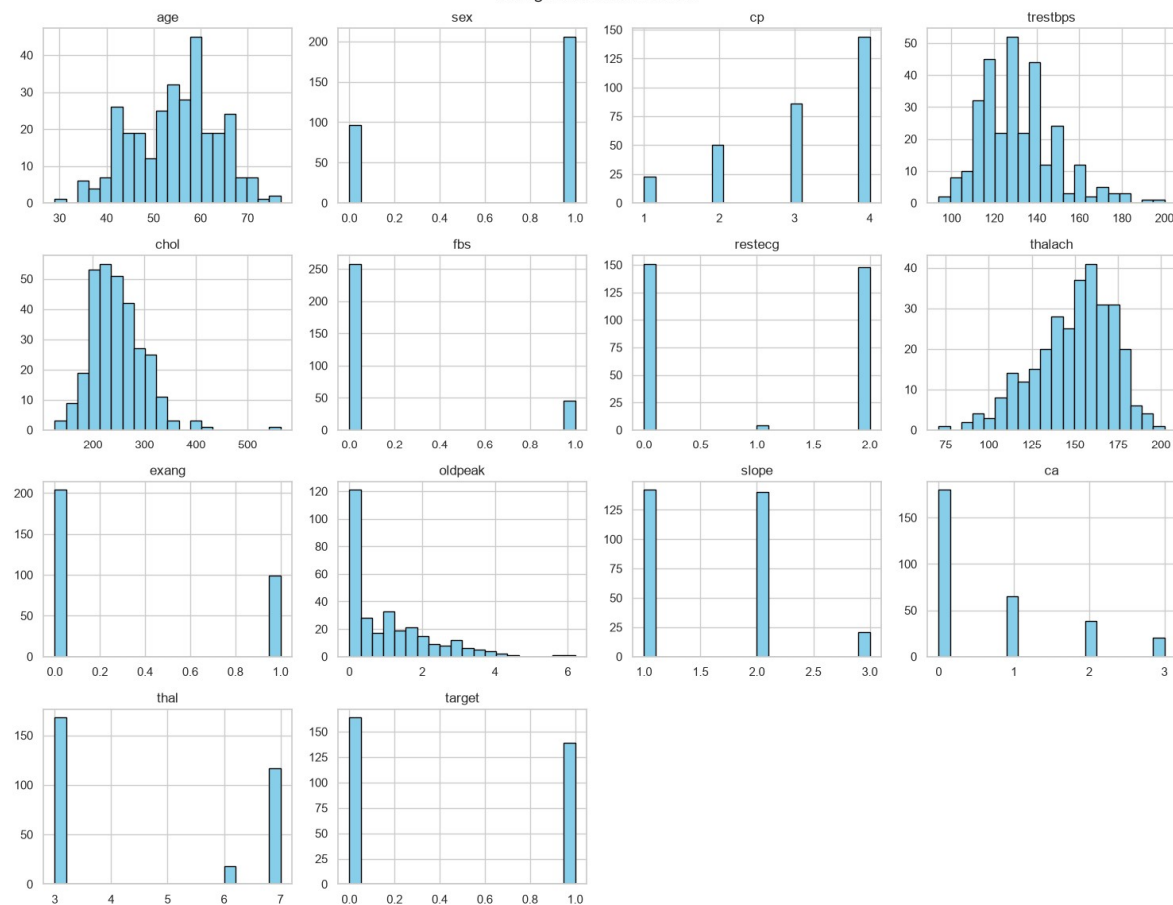
Exploratory Data Analysis (EDA) & Data Cleaning

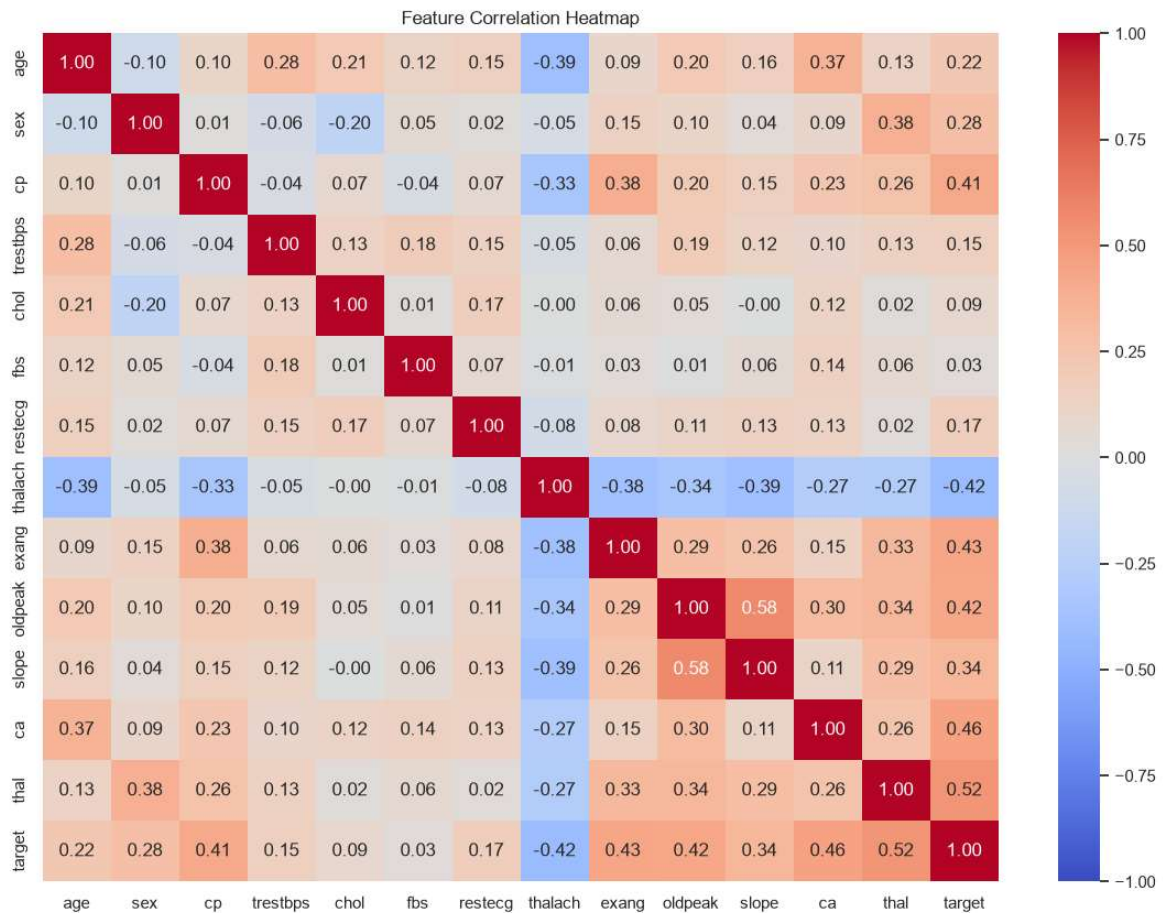
The dataset used is the UCI Heart Disease dataset. During the EDA and cleaning phase (documented in notebooks/01_eda_and_cleaning.ipynb and automated in src/download_data.py), the following steps were taken:

- **Target Variable Transformation:** The original target variable contained multiple classes representing the severity of heart disease. To frame this as a standard binary classification problem, the target was binarized (0 = no disease, 1 = presence of disease).
- **Handling Missing Values:** Missing values were identified in the dataset. To prevent data loss and maintain the distribution, missing continuous values were imputed using the median of their respective columns.



Histograms of All Features





Modelling Choices

- **Feature Selection:** All available physiological features (e.g., age, cholesterol, resting blood pressure, max heart rate) were utilized as they are clinically relevant to heart disease prediction.
- **Model Selection:** An automated champion/challenger approach was implemented. Both a Logistic Regression model (to establish a highly interpretable baseline) and a Random Forest Classifier (to capture complex, non-linear relationships) were trained within a pipeline containing a StandardScaler. The system automatically compared their performance using the ROC-AUC metric via MLflow, and dynamically selected the best-performing model to be serialized for the production API. Evaluation: The model's performance was evaluated using standard classification metrics, ensuring a balance between precision and recall, which is critical in healthcare diagnostics.

6. Experiment Tracking Summary

Model training runs and hyperparameters were tracked using **MLflow**.

- A local SQLite database (mlflow.db) was utilized as the tracking backend to log experiment data.

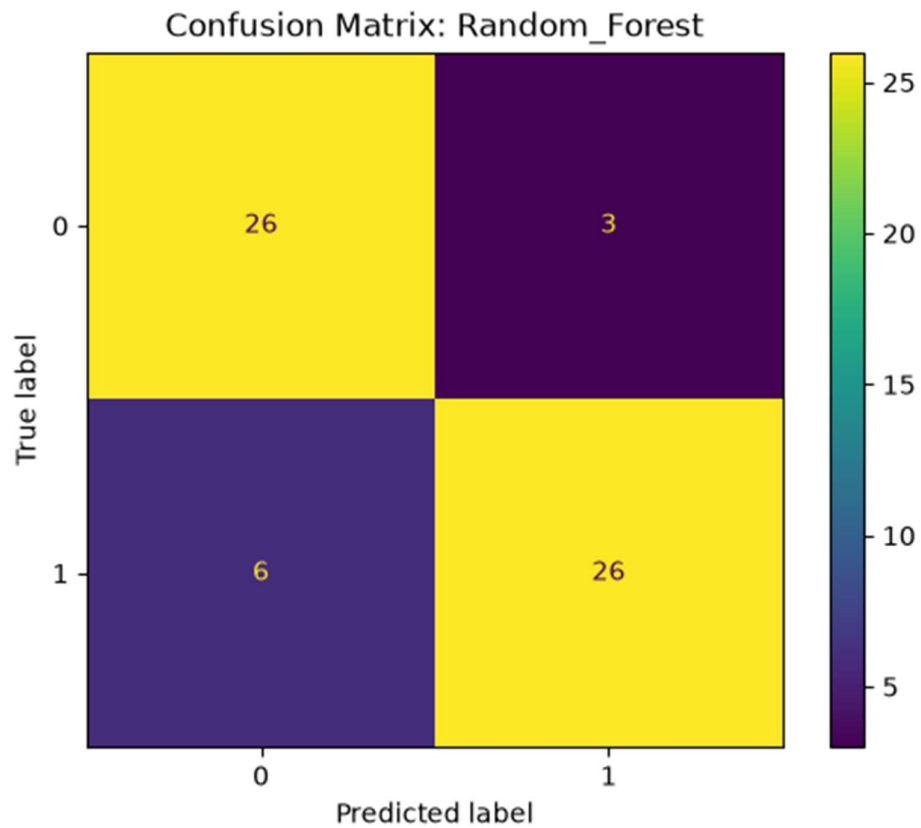
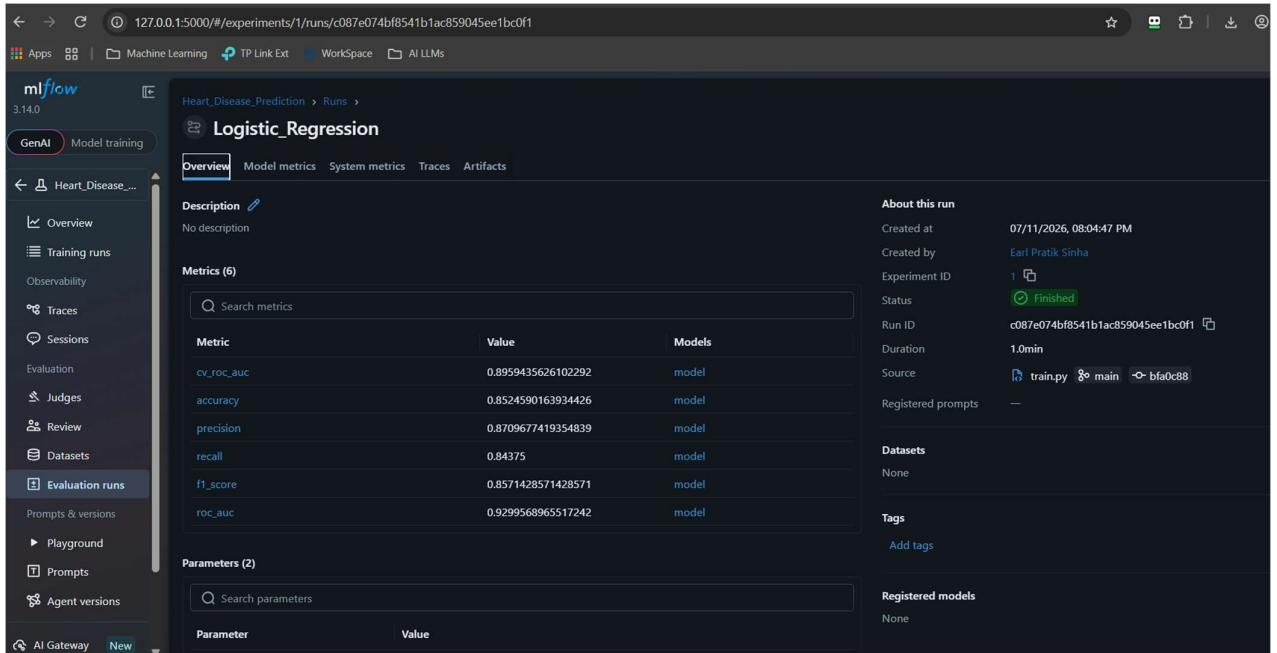
- MLflow tracked experiment parameters, evaluation metrics (Accuracy, Precision, Recall, F1-score and ROC-AUC), the trained model, and artifacts including ROC Curve and Confusion Matrix visualizations.
- The system automatically identifies the best-performing model from these tracked runs and serializes it as `models/best_model.pkl` to be served by the FastAPI application.

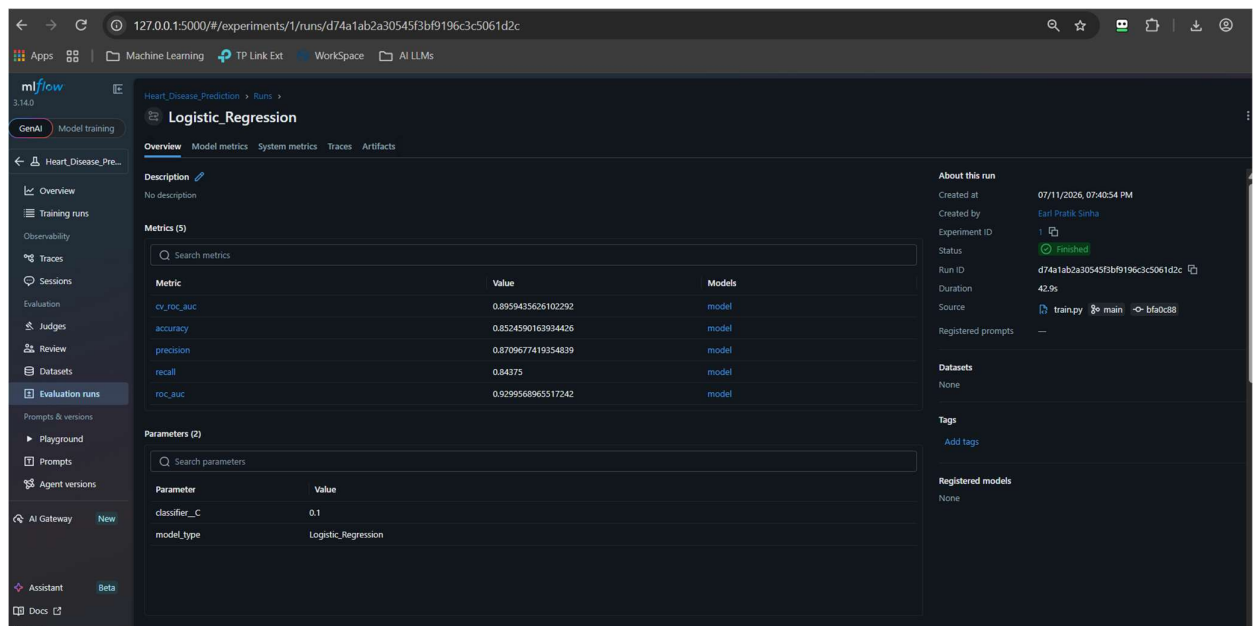
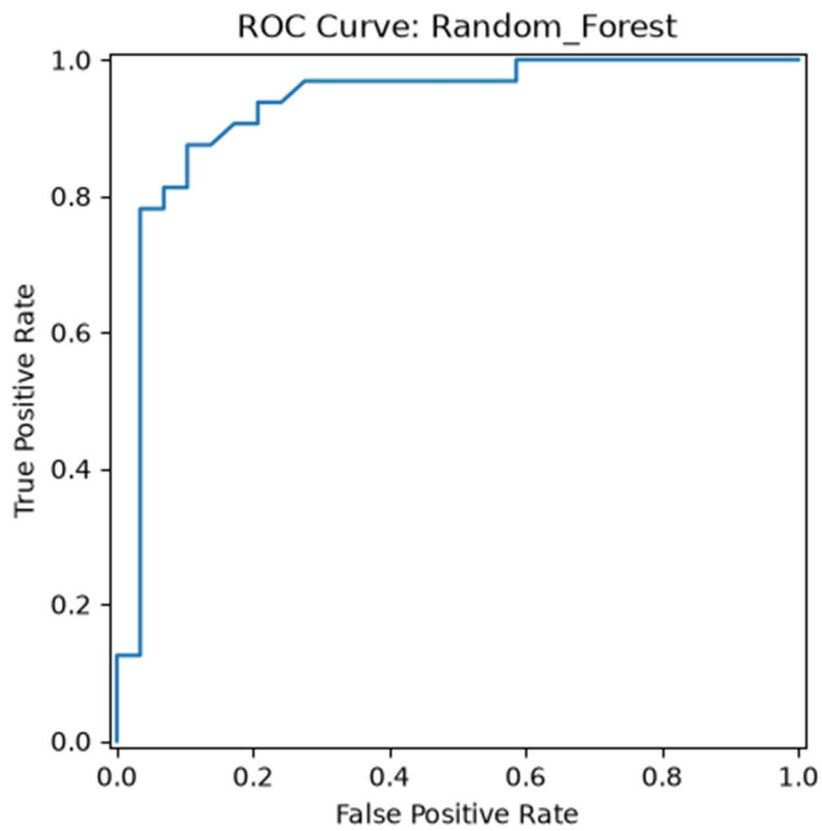
The screenshot shows the MLflow Training runs interface for the experiment 'Heart_Disease_Prediction'. The interface includes a sidebar with navigation options like Overview, Training runs, and Evaluation runs. The main area displays a table of training runs with columns for Run Name, Created, Dataset, Duration, Source, and Models. The runs are filtered by 'metrics.rmse < 1 and params.model = "tree"'. The runs are sorted by 'Created' time, showing the most recent runs at the top.

Run Name	Created	Dataset	Duration	Source	Models
Random_Forest	5 minutes ago	-	19.0s	train.py	model
Logistic_Regression	6 minutes ago	-	1.0min	train.py	model
Random_Forest	29 minutes ago	-	16.8s	train.py	model
Logistic_Regression	30 minutes ago	-	42.9s	train.py	model
Random_Forest	2 hours ago	-	14.0s	src/train...	model
Logistic_Regression	2 hours ago	-	19.4s	src/train...	model

The screenshot shows the MLflow Evaluation runs interface for the experiment 'Heart_Disease_Prediction'. The interface displays the details of a specific run named 'Random_Forest'. The 'Overview' tab is selected, showing the run's description, metrics, parameters, and artifacts. The 'About this run' section provides information about the run's creation, status, and duration. The 'Metrics' section lists the evaluation metrics for the run, including cv_roc_auc, accuracy, precision, recall, f1_score, and roc_auc. The 'Parameters' section lists the parameters used for the run, including cv_roc_auc, accuracy, precision, recall, f1_score, and roc_auc.

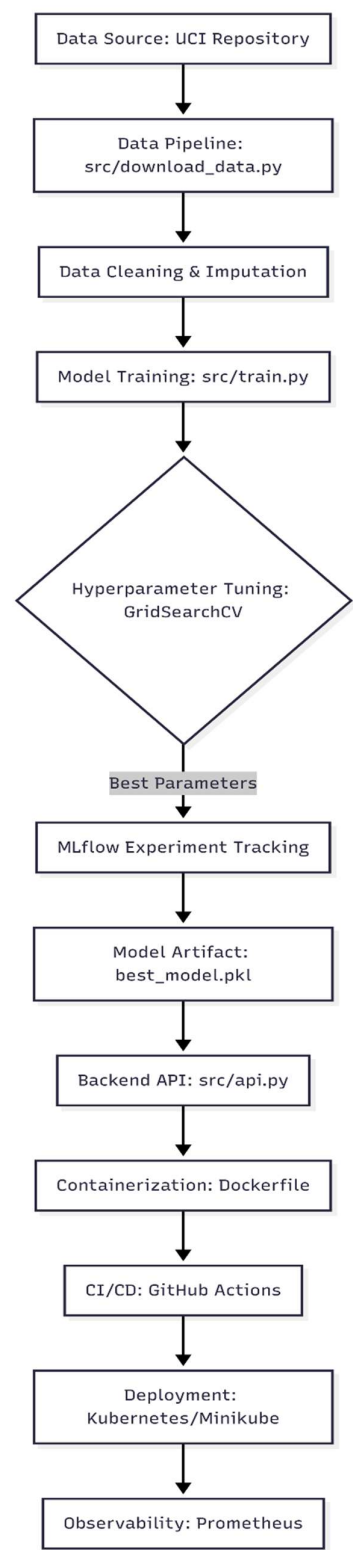
Metric	Value	Models
cv_roc_auc	0.8893458393458393	model
accuracy	0.8524590163934426	model
precision	0.896551724137931	model
recall	0.8125	model
f1_score	0.8524590163934426	model
roc_auc	0.9310344827586207	model



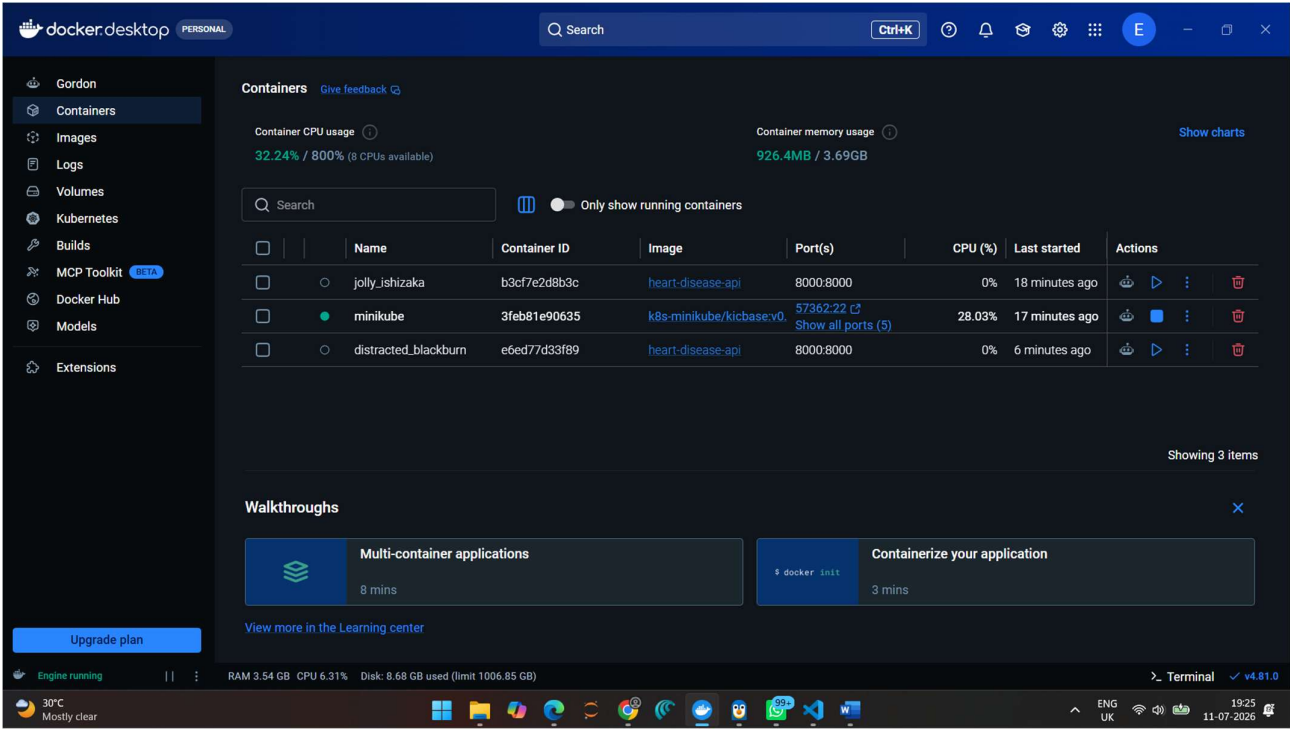


7. Architecture Diagram

Below is the high-level architecture of the MLOps pipeline:



Docker –



Video link - <https://drive.google.com/file/d/1H8ercOb-Ap6BFAoywdTbK4sRwrROROdS/view?usp=sharing>

Git - <https://github.com/earlpratiksinha/heart-disease-mlops>